

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Experiments (High)
Assessment Tool Weightage: 40%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5

7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I Darshinie Karani.V.M (192520033)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation
---------------	---	--	--	----------------------------------

ANSWERS

1.

Creating a database for Student_course

```
mysql> CREATE DATABASE student_course_db;
Query OK, 1 row affected (0.01 sec)

mysql> USE student_course_db;
Database changed
mysql> CREATE TABLE STUDENT_COURSE (
  ->     SID INT,
  ->     SName VARCHAR(50),
  ->     CID INT,
  ->     CName VARCHAR(50),
  ->     Instructor VARCHAR(50),
  ->     Grade VARCHAR(5),
  ->     PRIMARY KEY (SID, CID)
  -> );
Query OK, 0 rows affected (0.07 sec)
```

Inserting the values

```
mysql> INSERT INTO STUDENT_COURSE VALUES
  -> (101,'Arun',201,'DBMS','Kumar','A'),
  -> (101,'Arun',202,'Python','Ravi','B+'),
  -> (102,'Priya',201,'DBMS','Kumar','A+'),
  -> (102,'Priya',203,'Java','Meena','B'),
  -> (103,'Rahul',201,'DBMS','Kumar','B+');
Query OK, 5 rows affected (0.04 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

To view the table

```
mysql> SELECT * FROM STUDENT_COURSE;
+----+-----+-----+-----+-----+-----+
| SID | SName | CID | CName | Instructor | Grade |
+----+-----+-----+-----+-----+-----+
| 101 | Arun  | 201 | DBMS  | Kumar      | A      |
| 101 | Arun  | 202 | Python | Ravi       | B+     |
| 102 | Priya | 201 | DBMS  | Kumar      | A+     |
| 102 | Priya | 203 | Java  | Meena      | B      |
| 103 | Rahul | 201 | DBMS  | Kumar      | B+     |
+----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Using the SQL Query

```
mysql> SELECT SID, COUNT(DISTINCT SName) AS Name_Count
-> FROM STUDENT_COURSE
-> GROUP BY SID;
+----+-----+
| SID | Name_Count |
+----+-----+
| 101 | 1 |
| 102 | 1 |
| 103 | 1 |
+----+-----+
3 rows in set (0.04 sec)

mysql> SELECT CID,
COUNT(DISTINCT CName) AS Course_Count,
COUNT(DISTINCT Instructor) AS Instructor_Count
-> FROM STUDENT_COURSE
-> GROUP BY CID;
+----+-----+-----+
| CID | Course_Count | Instructor_Count |
+----+-----+-----+
| 201 | 1 | 1 |
| 202 | 1 | 1 |
| 203 | 1 | 1 |
+----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT SID, CID, COUNT(DISTINCT Grade) AS Grade_Count
-> FROM STUDENT_COURSE
-> GROUP BY SID, CID;
+----+----+-----+
| SID | CID | Grade_Count |
+----+----+-----+
| 101 | 201 | 1 |
| 101 | 202 | 1 |
| 102 | 201 | 1 |
| 102 | 203 | 1 |
| 103 | 201 | 1 |
+----+----+-----+
5 rows in set (0.00 sec)

mysql> SELECT SID, CID, COUNT(*)
-> FROM STUDENT_COURSE
-> GROUP BY SID, CID
-> HAVING COUNT(*) > 1;
Empty set (0.00 sec)

mysql> |
```

2.

Create a database

```
mysql> USE normalization_db;
Database changed
mysql> CREATE TABLE student_unf (
->     student_id INT,
->     student_name VARCHAR(50),
->     courses VARCHAR(100)
-> );
ERROR 1050 (42S01): Table 'student_unf' already exists
```

Insert the values

```
mysql> INSERT INTO student_unf VALUES
-> (101, 'Alice', 'DBMS, OS, CN'),
-> (102, 'Bob', 'Java, Python'),
-> (103, 'Charlie', 'DBMS');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

Select the table

```
mysql> SELECT * FROM student_unf;
+----+-----+-----+
| SID | SName   | Courses      |
+----+-----+-----+
| 101 | Alice   | DBMS, OS, CN |
| 102 | Bob     | Java, Python |
| 103 | Charlie | DBMS         |
+----+-----+-----+
3 rows in set (0.00 sec)
```

Create the 1NF Table

```
mysql> CREATE TABLE student_1nf (  
->     student_id INT,  
->     student_name VARCHAR(50),  
->     course VARCHAR(50),  
->     PRIMARY KEY (student_id, course)  
-> );  
Query OK, 0 rows affected (0.02 sec)
```

Insert the values

```
mysql> INSERT INTO student_1nf VALUES  
-> (101, 'Alice', 'DBMS'),  
-> (101, 'Alice', 'OS'),  
-> (101, 'Alice', 'CN'),  
-> (102, 'Bob', 'Java'),  
-> (102, 'Bob', 'Python'),  
-> (103, 'Charlie', 'DBMS');  
Query OK, 6 rows affected (0.01 sec)  
Records: 6  Duplicates: 0  Warnings: 0
```

View the table

```
mysql> SELECT * FROM student_1nf;  
+-----+-----+-----+  
| student_id | student_name | course |  
+-----+-----+-----+  
|          101 | Alice        | CN     |  
|          101 | Alice        | DBMS   |  
|          101 | Alice        | OS     |  
|          102 | Bob          | Java   |  
|          102 | Bob          | Python |  
|          103 | Charlie      | DBMS   |  
+-----+-----+-----+  
6 rows in set (0.00 sec)
```

3.

Create database

```
mysql> CREATE DATABASE normalization_db;  
ERROR 1007 (HY000): Can't create database 'normalization_db'; database exists  
mysql> USE normalization_db;  
Database changed
```

Create relation R

```
mysql> CREATE TABLE R (  
->     A INT,  
->     B VARCHAR(20),  
->     C VARCHAR(20),  
->     D VARCHAR(20),  
->     E VARCHAR(20),  
->     PRIMARY KEY (A)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

Insert the sample data

```
mysql> INSERT INTO R VALUES
      -> (1, 'B1', 'C1', 'D1', 'E1'),
      -> (2, 'B2', 'C2', 'D2', 'E2'),
      -> (3, 'B3', 'C3', 'D3', 'E3');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Display the relation

```
mysql> SELECT * FROM R;
+---+-----+-----+-----+-----+
| A | B      | C      | D      | E      |
+---+-----+-----+-----+-----+
| 1 | B1     | C1     | D1     | E1     |
| 2 | B2     | C2     | D2     | E2     |
| 3 | B3     | C3     | D3     | E3     |
+---+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Show that A determines B, C

```
mysql> SELECT A, B, C
      -> FROM R;
+---+-----+-----+
| A | B      | C      |
+---+-----+-----+
| 1 | B1     | C1     |
| 2 | B2     | C2     |
| 3 | B3     | C3     |
+---+-----+-----+
3 rows in set (0.00 sec)
```

$A \rightarrow BC$

Show $BC \rightarrow D$

```
mysql> SELECT B, C, D
      -> FROM R;
+-----+-----+-----+
| B      | C      | D      |
+-----+-----+-----+
| B1     | C1     | D1     |
| B2     | C2     | D2     |
| B3     | C3     | D3     |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

$BC \rightarrow D$

Show $D \rightarrow E$

```
mysql> SELECT D, E
-> FROM R;
+-----+-----+
| D     | E     |
+-----+-----+
| D1    | E1    |
| D2    | E2    |
| D3    | E3    |
+-----+-----+
3 rows in set (0.00 sec)
```

$D \rightarrow E$

4.

Create database

```
mysql> CREATE DATABASE employee_db;
Query OK, 1 row affected (0.01 sec)

mysql> USE employee_db;
Database changed
```

Create original relation

```
mysql> CREATE TABLE EMPLOYEE (
->     EmpID INT PRIMARY KEY,
->     EmpName VARCHAR(50),
->     DeptID INT,
->     DeptName VARCHAR(50),
->     ManagerID INT
-> );
Query OK, 0 rows affected (0.02 sec)
```

Insert the sample data

```
mysql> INSERT INTO EMPLOYEE VALUES
-> (101, 'Alice', 10, 'HR', 501),
-> (102, 'Bob', 20, 'IT', 502),
-> (103, 'Charlie', 10, 'HR', 501),
-> (104, 'David', 30, 'Finance', 503);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```


Decompose into 3NF

```
mysql> CREATE TABLE EMPLOYEE_3NF (  
->     EmpID INT PRIMARY KEY,  
->     EmpName VARCHAR(50),  
->     DeptID INT  
-> );  
Query OK, 0 rows affected (0.08 sec)  
  
mysql> CREATE TABLE DEPARTMENT (  
->     DeptID INT PRIMARY KEY,  
->     DeptName VARCHAR(50),  
->     ManagerID INT  
-> );  
Query OK, 0 rows affected (0.02 sec)
```

Insert data into decomposed tables

```
mysql> INSERT INTO EMPLOYEE_3NF VALUES  
-> (101, 'Alice', 10),  
-> (102, 'Bob', 20),  
-> (103, 'Charlie', 10),  
-> (104, 'David', 30);  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0  
  
mysql> INSERT INTO DEPARTMENT VALUES  
-> (10, 'HR', 501),  
-> (20, 'IT', 502),  
-> (30, 'Finance', 503);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

Add foreign key

```
mysql> ALTER TABLE EMPLOYEE_3NF  
-> ADD FOREIGN KEY (DeptID)  
-> REFERENCES DEPARTMENT(DeptID);  
Query OK, 4 rows affected (0.05 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

Display the 3NF tables

```
mysql> SELECT * FROM EMPLOYEE_3NF;  
+-----+-----+-----+  
| EmpID | EmpName | DeptID |  
+-----+-----+-----+  
| 101 | Alice | 10 |  
| 102 | Bob | 20 |  
| 103 | Charlie | 10 |  
| 104 | David | 30 |  
+-----+-----+-----+  
4 rows in set (0.00 sec)  
  
mysql>  
mysql> SELECT * FROM DEPARTMENT;  
+-----+-----+-----+  
| DeptID | DeptName | ManagerID |  
+-----+-----+-----+  
| 10 | HR | 501 |  
| 20 | IT | 502 |  
| 30 | Finance | 503 |  
+-----+-----+-----+  
3 rows in set (0.00 sec)
```

Join to get the original information

```
mysql> SELECT
->     E.EmpID,
->     E.EmpName,
->     E.DeptID,
->     D.DeptName,
->     D.ManagerID
-> FROM EMPLOYEE_3NF E
-> JOIN DEPARTMENT D
-> ON E.DeptID = D.DeptID;
+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-----+-----+-----+-----+-----+
| 101   | Alice   | 10     | HR       | 501       |
| 103   | Charlie | 10     | HR       | 501       |
| 102   | Bob     | 20     | IT       | 502       |
| 104   | David   | 30     | Finance  | 503       |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

5.

Create database

```
mysql> CREATE DATABASE bcnf_db;
Query OK, 1 row affected (0.01 sec)

mysql> USE bcnf_db;
Database changed
```

Create original relation

```
mysql> CREATE TABLE R (
->     A VARCHAR(20),
->     B VARCHAR(20),
->     C VARCHAR(20),
->     D VARCHAR(20),
->     PRIMARY KEY (A, B)
-> );
Query OK, 0 rows affected (0.13 sec)
```

Insert sample data

```
mysql> INSERT INTO R VALUES
-> ('A1', 'B1', 'C1', 'D1'),
-> ('A2', 'B2', 'C2', 'D2'),
-> ('A3', 'B3', 'C1', 'D1');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Check the functional dependency $C \rightarrow D$

```
mysql> SELECT C, COUNT(DISTINCT D) AS D_values
-> FROM R
-> GROUP BY C;
+-----+-----+
| C   | D_values |
+-----+-----+
| C1  | 1       |
| C2  | 1       |
+-----+-----+
2 rows in set (0.00 sec)
```

BCNF Decomposition

```
mysql> CREATE TABLE R1 (  
    ->     C VARCHAR(20) PRIMARY KEY,  
    ->     D VARCHAR(20)  
    -> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> CREATE TABLE R2 (  
    ->     A VARCHAR(20),  
    ->     B VARCHAR(20),  
    ->     C VARCHAR(20),  
    ->     PRIMARY KEY (A, B)  
    -> );  
Query OK, 0 rows affected (0.02 sec)
```

Insert data into decomposed relations

```
mysql> INSERT INTO R1 VALUES  
    -> ('C1', 'D1'),  
    -> ('C2', 'D2');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2  Duplicates: 0  Warnings: 0  
  
mysql> INSERT INTO R2 VALUES  
    -> ('A1', 'B1', 'C1'),  
    -> ('A2', 'B2', 'C2'),  
    -> ('A3', 'B3', 'C1');  
Query OK, 3 rows affected (0.06 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

Verify the decomposed tables

```
mysql> SELECT * FROM R1;  
+----+-----+  
| C  | D    |  
+----+-----+  
| C1 | D1   |  
| C2 | D2   |  
+----+-----+  
2 rows in set (0.00 sec)  
  
mysql> SELECT * FROM R2;  
+----+-----+-----+  
| A  | B    | C    |  
+----+-----+-----+  
| A1 | B1   | C1   |  
| A2 | B2   | C2   |  
| A3 | B3   | C1   |  
+----+-----+-----+  
3 rows in set (0.00 sec)
```

Reconstruct original relation

```
mysql> SELECT
->     R2.A,
->     R2.B,
->     R2.C,
->     R1.D
-> FROM R2
-> JOIN R1
-> ON R2.C = R1.C;
+----+----+-----+-----+
| A  | B  | C     | D     |
+----+----+-----+-----+
| A1 | B1 | C1    | D1    |
| A2 | B2 | C2    | D2    |
| A3 | B3 | C1    | D1    |
+----+----+-----+-----+
3 rows in set (0.00 sec)
```

6.

Create database

```
mysql> CREATE DATABASE chase_db;
Query OK, 1 row affected (0.01 sec)

mysql> USE chase_db;
Database changed
```

Create original relation

```
mysql> CREATE TABLE R (
->     A INT,
->     B VARCHAR(20),
->     C VARCHAR(20),
->     PRIMARY KEY (A)
-> );
Query OK, 0 rows affected (0.07 sec)
```

Insert sample data

```
mysql> INSERT INTO R VALUES
-> (1, 'B1', 'C1'),
-> (2, 'B2', 'C2'),
-> (3, 'B3', 'C3');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Create decomposed relations

```
mysql> CREATE TABLE R1 (  
->     A INT PRIMARY KEY,  
->     B VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.06 sec)  
  
mysql> CREATE TABLE R2 (  
->     A INT PRIMARY KEY,  
->     C VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.12 sec)
```

Insert the decomposed data

```
mysql> INSERT INTO R1  
-> SELECT A, B  
-> FROM R;  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> INSERT INTO R2  
-> SELECT A, C  
-> FROM R;  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

Perform the JOIN

```
mysql> SELECT  
->     R1.A,  
->     R1.B,  
->     R2.C  
-> FROM R1  
-> JOIN R2  
-> ON R1.A = R2.A;  
  
+---+-----+-----+  
| A | B      | C      |  
+---+-----+-----+  
| 1 | B1     | C1     |  
| 2 | B2     | C2     |  
| 3 | B3     | C3     |  
+---+-----+-----+  
3 rows in set (0.00 sec)
```

Verify lossless join automatically

```
mysql> SELECT * FROM R
->
-> EXCEPT
->
-> SELECT
->     R1.A,
->     R1.B,
->     R2.C
-> FROM R1
-> JOIN R2
-> ON R1.A = R2.A;
Empty set (0.00 sec)
```

7.

Create database

```
mysql> CREATE DATABASE four_nf_db;
Query OK, 1 row affected (0.04 sec)

mysql> USE four_nf_db;
Database changed
```

Create the original TEACHER table

```
mysql> CREATE TABLE TEACHER (
->     TID INT,
->     Subject VARCHAR(50),
->     Hobby VARCHAR(50),
->     PRIMARY KEY (TID, Subject, Hobby)
-> );
Query OK, 0 rows affected (0.06 sec)
```

Insert sample data

```
mysql> INSERT INTO TEACHER VALUES
-> (101, 'DBMS', 'Drawing'),
-> (101, 'DBMS', 'Singing'),
-> (101, 'OS', 'Drawing'),
-> (101, 'OS', 'Singing'),
-> (102, 'Java', 'Cricket'),
-> (102, 'Python', 'Cricket');
Query OK, 6 rows affected (0.04 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM TEACHER;
+-----+-----+-----+
| TID | Subject | Hobby |
+-----+-----+-----+
| 101 | DBMS    | Drawing |
| 101 | DBMS    | Singing |
| 101 | OS      | Drawing |
| 101 | OS      | Singing |
| 102 | Java    | Cricket |
| 102 | Python  | Cricket |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

Decompose into 4NF

```
mysql> CREATE TABLE TEACHER_SUBJECT (
->     TID INT,
->     Subject VARCHAR(50),
->     PRIMARY KEY (TID, Subject)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE TEACHER_HOBBY (
->     TID INT,
->     Hobby VARCHAR(50),
->     PRIMARY KEY (TID, Hobby)
-> );
Query OK, 0 rows affected (0.06 sec)
```

Insert data into decomposed tables

```
mysql> INSERT INTO TEACHER_SUBJECT
-> SELECT DISTINCT TID, Subject
-> FROM TEACHER;
Query OK, 4 rows affected (0.04 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO TEACHER_HOBBY
-> SELECT DISTINCT TID, Hobby
-> FROM TEACHER;
Query OK, 3 rows affected (0.04 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Display the 4NF relations

```
mysql> SELECT * FROM TEACHER_SUBJECT;
+-----+-----+
| TID | Subject |
+-----+-----+
| 101 | DBMS    |
| 101 | OS      |
| 102 | Java    |
| 102 | Python  |
+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM TEACHER_HOBBY;
+-----+-----+
| TID | Hobby   |
+-----+-----+
| 101 | Drawing |
| 101 | Singing |
| 102 | Cricket |
+-----+-----+
3 rows in set (0.00 sec)
```

Reconstruct the original relation

```
mysql> SELECT
->     S.TID,
->     S.Subject,
->     H.Hobby
-> FROM TEACHER_SUBJECT S
-> JOIN TEACHER_HOBBY H
-> ON S.TID = H.TID;
+-----+-----+-----+
| TID | Subject | Hobby   |
+-----+-----+-----+
| 101 | DBMS    | Drawing |
| 101 | OS      | Drawing |
| 101 | DBMS    | Singing |
| 101 | OS      | Singing |
| 102 | Java    | Cricket |
| 102 | Python  | Cricket |
+-----+-----+-----+
6 rows in set (0.00 sec)
```


8.

Create database

```
mysql> CREATE DATABASE five_nf_db;
Query OK, 1 row affected (0.01 sec)

mysql> USE five_nf_db;
Database changed
```

Create original relation

```
mysql> CREATE TABLE SPJ (
->     Supplier VARCHAR(20),
->     Part VARCHAR(20),
->     Project VARCHAR(20),
->     PRIMARY KEY (Supplier, Part, Project)
-> );
Query OK, 0 rows affected (0.03 sec)
```

Insert sample data

```
mysql> INSERT INTO SPJ VALUES
-> ('S1', 'P1', 'J1'),
-> ('S1', 'P2', 'J1'),
-> ('S2', 'P1', 'J2'),
-> ('S2', 'P2', 'J2');
Query OK, 4 rows affected (0.04 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM SPJ;
+-----+-----+-----+
| Supplier | Part | Project |
+-----+-----+-----+
| S1       | P1   | J1      |
| S1       | P2   | J1      |
| S2       | P1   | J2      |
| S2       | P2   | J2      |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

Decompose into 5NF relations for supplier-part, supplier-project and part-project

```
mysql> CREATE TABLE SP (
->     Supplier VARCHAR(20),
->     Part VARCHAR(20),
->     PRIMARY KEY (Supplier, Part)
-> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE SJ (
->     Supplier VARCHAR(20),
->     Project VARCHAR(20),
->     PRIMARY KEY (Supplier, Project)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE PJ (
->     Part VARCHAR(20),
->     Project VARCHAR(20),
->     PRIMARY KEY (Part, Project)
-> );
Query OK, 0 rows affected (0.06 sec)
```

Insert data

```
mysql> INSERT INTO SP
-> SELECT DISTINCT Supplier, Part
-> FROM SPJ;
Query OK, 4 rows affected (0.04 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO SJ
-> SELECT DISTINCT Supplier, Project
-> FROM SPJ;
Query OK, 2 rows affected (0.04 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO PJ
-> SELECT DISTINCT Part, Project
-> FROM SPJ;
Query OK, 4 rows affected (0.04 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Display the decomposed relations

```
mysql> SELECT * FROM SP;
+-----+-----+
| Supplier | Part |
+-----+-----+
| S1       | P1   |
| S1       | P2   |
| S2       | P1   |
| S2       | P2   |
+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM SJ;
+-----+-----+
| Supplier | Project |
+-----+-----+
| S1       | J1      |
| S2       | J2      |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM PJ;
+-----+-----+
| Part | Project |
+-----+-----+
| P1   | J1      |
| P1   | J2      |
| P2   | J1      |
| P2   | J2      |
+-----+-----+
4 rows in set (0.00 sec)
```

Reconstruct using JOIN

```
mysql> SELECT
-> SP.Supplier,
-> SP.Part,
-> SJ.Project
-> FROM SP
-> JOIN SJ
-> ON SP.Supplier = SJ.Supplier
-> JOIN PJ
-> ON SP.Part = PJ.Part
-> AND SJ.Project = PJ.Project;
+-----+-----+-----+
| Supplier | Part | Project |
+-----+-----+-----+
| S1       | P1   | J1      |
| S1       | P2   | J1      |
| S2       | P1   | J2      |
| S2       | P2   | J2      |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

Check for redundancy / extra tuples

```
mysql> SELECT
->     SP.Supplier,
->     SP.Part,
->     SJ.Project
-> FROM SP
-> JOIN SJ
->     ON SP.Supplier = SJ.Supplier
-> JOIN PJ
->     ON SP.Part = PJ.Part
->     AND SJ.Project = PJ.Project
->
-> WHERE NOT EXISTS (
->     SELECT 1
->     FROM SPJ
->     WHERE SPJ.Supplier = SP.Supplier
->           AND SPJ.Part = SP.Part
->           AND SPJ.Project = SJ.Project
-> );
Empty set (0.00 sec)
```

9.

Create database

```
mysql> CREATE DATABASE college_db;
Query OK, 1 row affected (0.05 sec)

mysql> USE college_db;
Database changed
```

Create the poorly designed table

```
mysql> CREATE TABLE COLLEGE (
->     StudentID INT,
->     StudentName VARCHAR(50),
->     CourseID INT,
->     CourseName VARCHAR(50),
->     InstructorID INT,
->     InstructorName VARCHAR(50),
->     InstructorPhone VARCHAR(15)
-> );
Query OK, 0 rows affected (0.12 sec)
```

Insert sample data

```
mysql> INSERT INTO COLLEGE VALUES
-> (101, 'Alice', 1, 'DBMS', 501, 'Dr. Kumar', '9876543210'),
-> (102, 'Bob', 1, 'DBMS', 501, 'Dr. Kumar', '9876543210'),
-> (103, 'Charlie', 2, 'OS', 502, 'Dr. Ravi', '9876543211'),
-> (104, 'David', 2, 'OS', 502, 'Dr. Ravi', '9876543211');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM COLLEGE;
+-----+-----+-----+-----+-----+-----+-----+
| StudentID | StudentName | CourseID | CourseName | InstructorID | InstructorName | InstructorPhone |
+-----+-----+-----+-----+-----+-----+-----+
| 101 | Alice | 1 | DBMS | 501 | Dr. Kumar | 9876543210 |
| 102 | Bob | 1 | DBMS | 501 | Dr. Kumar | 9876543210 |
| 103 | Charlie | 2 | OS | 502 | Dr. Ravi | 9876543211 |
| 104 | David | 2 | OS | 502 | Dr. Ravi | 9876543211 |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Demonstrate Update Anomaly

```
mysql> UPDATE COLLEGE
-> SET InstructorPhone = '9999999999'
-> WHERE InstructorID = 501;
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

```
mysql> SELECT * FROM COLLEGE;
```

StudentID	StudentName	CourseID	CourseName	InstructorID	InstructorName	InstructorPhone
101	Alice	1	DBMS	501	Dr. Kumar	9999999999
102	Bob	1	DBMS	501	Dr. Kumar	9999999999
103	Charlie	2	OS	502	Dr. Ravi	9876543211
104	David	2	OS	502	Dr. Ravi	9876543211

4 rows in set (0.00 sec)

Demonstrate Insertion Anomaly

```
mysql> INSERT INTO COLLEGE
-> VALUES (NULL, NULL, 3, 'Computer Networks', 503, 'Dr. Priya', '9876543212');
Query OK, 1 row affected (0.04 sec)
```

Demonstrate Deletion Anomaly

```
mysql> DELETE FROM COLLEGE
-> WHERE StudentID = 103;
Query OK, 1 row affected (0.04 sec)
```

Create STUDENT table

```
mysql> CREATE TABLE STUDENT (
->     StudentID INT PRIMARY KEY,
->     StudentName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.06 sec)
```

Create INSTRUCTOR table

```
mysql> CREATE TABLE INSTRUCTOR (
->     InstructorID INT PRIMARY KEY,
->     InstructorName VARCHAR(50),
->     InstructorPhone VARCHAR(15)
-> );
Query OK, 0 rows affected (0.06 sec)
```

Create COURSE table

```
mysql> CREATE TABLE COURSE (
->     CourseID INT PRIMARY KEY,
->     CourseName VARCHAR(50),
->     InstructorID INT,
->     FOREIGN KEY (InstructorID)
->     REFERENCES INSTRUCTOR(InstructorID)
-> );
Query OK, 0 rows affected (0.07 sec)
```

Create ENROLLMENT table

```
mysql> CREATE TABLE ENROLLMENT (
->     StudentID INT,
->     CourseID INT,
->     PRIMARY KEY (StudentID, CourseID),
->     FOREIGN KEY (StudentID)
->     REFERENCES STUDENT(StudentID),
->     FOREIGN KEY (CourseID)
->     REFERENCES COURSE(CourseID)
-> );
Query OK, 0 rows affected (0.03 sec)
```

Insert data into normalized tables

```
mysql> INSERT INTO STUDENT VALUES
-> (101, 'Alice'),
-> (102, 'Bob'),
-> (103, 'Charlie'),
-> (104, 'David');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO INSTRUCTOR VALUES
-> (501, 'Dr. Kumar', '9999999999'),
-> (502, 'Dr. Ravi', '9876543211');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO COURSE VALUES
-> (1, 'DBMS', 501),
-> (2, 'OS', 502);
Query OK, 2 rows affected (0.04 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO ENROLLMENT VALUES
-> (101, 1),
-> (102, 1),
-> (103, 2),
-> (104, 2);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

Display the normalized data

```
mysql> SELECT * FROM STUDENT;
+-----+-----+
| StudentID | StudentName |
+-----+-----+
| 101 | Alice |
| 102 | Bob |
| 103 | Charlie |
| 104 | David |
+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM COURSE;
+-----+-----+-----+
| CourseID | CourseName | InstructorID |
+-----+-----+-----+
| 1 | DBMS | 501 |
| 2 | OS | 502 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM INSTRUCTOR;
+-----+-----+-----+
| InstructorID | InstructorName | InstructorPhone |
+-----+-----+-----+
| 501 | Dr. Kumar | 9999999999 |
| 502 | Dr. Ravi | 9876543211 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM ENROLLMENT;
+-----+-----+
| StudentID | CourseID |
+-----+-----+
| 101 | 1 |
| 102 | 1 |
| 103 | 2 |
| 104 | 2 |
+-----+-----+
4 rows in set (0.00 sec)
```

Reconstruct the complete information

```
mysql> SELECT
-> S.StudentID,
-> S.StudentName,
-> C.CourseID,
-> C.CourseName,
-> I.InstructorID,
-> I.InstructorName,
-> I.InstructorPhone
-> FROM STUDENT S
-> JOIN ENROLLMENT E
-> ON S.StudentID = E.StudentID
-> JOIN COURSE C
-> ON E.CourseID = C.CourseID
-> JOIN INSTRUCTOR I
-> ON C.InstructorID = I.InstructorID;
```

StudentID	StudentName	CourseID	CourseName	InstructorID	InstructorName	InstructorPhone
101	Alice	1	DBMS	501	Dr. Kumar	9999999999
102	Bob	1	DBMS	501	Dr. Kumar	9999999999
103	Charlie	2	OS	502	Dr. Ravi	9876543211
104	David	2	OS	502	Dr. Ravi	9876543211

4 rows in set (0.00 sec)

10.

Create database and table

```
mysql> CREATE DATABASE closure_db;
Query OK, 1 row affected (0.13 sec)
```

```
mysql> USE closure_db;
Database changed
```

```
mysql> CREATE TABLE R (
-> A VARCHAR(10),
-> B VARCHAR(10),
-> C VARCHAR(10),
-> D VARCHAR(10),
-> E VARCHAR(10)
-> );
Query OK, 0 rows affected (0.03 sec)
```

Insert sample data

```
mysql> INSERT INTO R VALUES
-> ('A1','B1','C1','D1','E1'),
-> ('A2','B2','C2','D2','E2'),
-> ('A3','B3','C3','D3','E3');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

Compute A⁺

```
mysql> SELECT A, B, C, D, E
-> FROM R;
```

A	B	C	D	E
A1	B1	C1	D1	E1
A2	B2	C2	D2	E2
A3	B3	C3	D3	E3

3 rows in set (0.00 sec)

Check whether B is a key

$$B^+ = \{B\}$$

It does not contain all attributes, so B is not a candidate key.

- Check C

$$C^+ = \{C\}$$

So C is not a candidate key.

- Check D

Given $D \rightarrow E$:

$$D^+ = \{D, E\}$$

So D is not a candidate key.

Check BC

Given:

$BC \rightarrow D$ and $D \rightarrow E$:

$$BC^+ = \{B, C, D, E\}$$

It does not contain A, so BC is not a candidate key.